

Lab 7: Phân tích cú pháp phụ thuộc (Dependency Parsing)

Phần 1: Giới thiệu và Cài đặt

```
# Cài đặt spaCy
! pip install -U spacy
# Tải về mô hình tiếng Anh (kích thước trung bình, có đủ thông tin cho parsing)
! python -m spacy download en_core_web_md

Requirement already satisfied: spacy in /usr/local/lib/python3.12/dist-packages (3.8.11)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.0.12)
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.5)
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.15)
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.13)
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.0.12)
Requirement already satisfied: thinc<8.4.0,>=8.3.4 in /usr/local/lib/python3.12/dist-packages (from spacy) (8.3.10)
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.1.3)
Requirement already satisfied: srsly<3.0.0,>=2.4.3 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.5.2)
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.10)
Requirement already satisfied: weasel<0.5.0,>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (0.4.3)
Requirement already satisfied: typer-slim<1.0.0,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (0.20.0)
Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (4.67.1)
Requirement already satisfied: numpy>=1.19.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.2)
Requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.32.4)
Requirement already satisfied: pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.1)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.1.6)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from spacy) (75.2.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (25.0)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4) (0.7.0)
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4) (2.41.4)
Requirement already satisfied: typing-extensions>=4.14.1 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4) (4.14.1)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4) (0.4.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0) (2025.1.1)
Requirement already satisfied: blis<1.4.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from thinc<8.4.0,>=8.3.4) (1.3.0)
Requirement already satisfied: confection<1.0.0,>=0.0.1 in /usr/local/lib/python3.12/dist-packages (from thinc<8.4.0,>=8.3.4) (0.1.5)
Requirement already satisfied: click>=8.0.0 in /usr/local/lib/python3.12/dist-packages (from typer-slim<1.0.0,>=0.3.0) (8.1.8)
Requirement already satisfied: cloudpathlib<1.0.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from weasel<0.5.0,>=0.4.2) (0.19.0)
Requirement already satisfied: smart-open<8.0.0,>=5.2.1 in /usr/local/lib/python3.12/dist-packages (from weasel<0.5.0,>=0.4.2) (7.0.5)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2) (3.0.2)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart-open<8.0.0,>=5.2.1) (1.16.0)
Collecting en-core-web-md==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en\_core\_web\_md-3.8.0/en\_core\_web\_md-3.8.0-py3-none-any.whl
33.5/33.5 MB 50.8 MB/s eta 0:00:00

Installing collected packages: en-core-web-md
Successfully installed en-core-web-md-3.8.0
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_md')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in order to load all the package's dependencies. You can do this by selecting the 'Restart kernel' or 'Restart runtime' option.
```

Phần 2: Phân tích câu và Trực quan hóa

```
import spacy
from spacy import displacy

# Tải mô hình tiếng Anh đã cài đặt
# Sử dụng en_core_web_md vì nó chứa các vector từ và cây cú pháp đầy đủ
nlp = spacy.load("en_core_web_md")
# Câu ví dụ
text = "The quick brown fox jumps over the lazy dog."
# Phân tích câu với pipeline của spaCy
doc = nlp(text)
doc

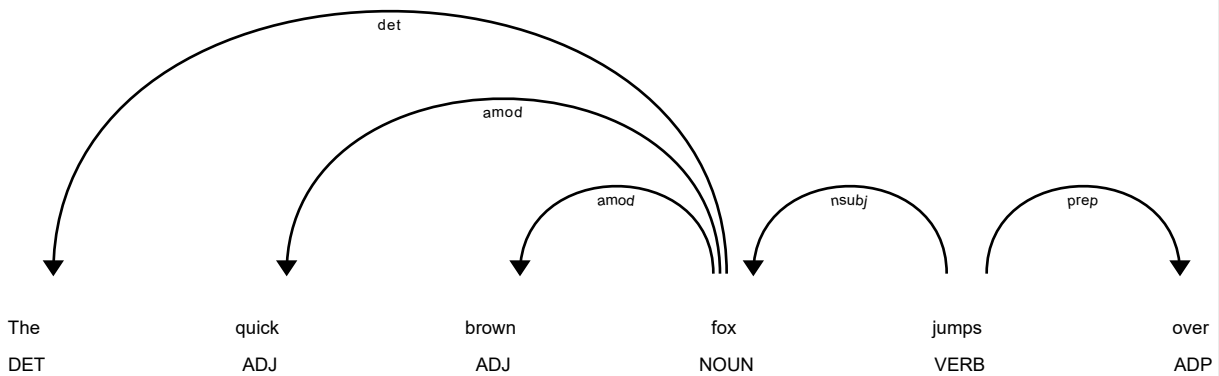
The quick brown fox jumps over the lazy dog.
```

Trực quan hóa cây phụ thuộc

- Từ nào là gốc (ROOT) của câu? --> jumps
- jumps có những từ phụ thuộc (dependent) nào? Các quan hệ đó là gì?
 - fox: nsubj
 - over: prep
- fox là head của những từ nào? --> the, quick, brown

```
# Tùy chọn để hiển thị trong trình duyệt
options = {"compact": True, "color": "blue", "font": "Source Sans Pro"}
# Khởi chạy server tại http://127.0.0.1:5000
displacy.serve(doc, style="dep")
```

/usr/local/lib/python3.12/dist-packages/spacy/displacy/__init__.py:108: UserWarning: [W011] It looks like you're calling displac warnings.warn(Warnings.W011)



Using the 'dep' visualizer
Serving on <http://0.0.0.0:5000> ...

Shutting down server on port 5000.

Phần 3: Truy cập các thành phần trong cây phụ thuộc

```
# Lấy một câu khác để phân tích
text = "Apple is looking at buying U.K. startup for $1 billion"
doc = nlp(text)
# In ra thông tin của từng token
print(f"{'TEXT':<12} | {'DEP':<10} | {'HEAD TEXT':<12} | {'HEAD POS':<8} | {'CHILDREN'}")
print("-" * 70)

for token in doc:
    # Trích xuất các thuộc tính
    children = [child.text for child in token.children]
    print(f"{token.text:<12} | {token.dep_:<10} | {token.head.text:<12} | {token.head.pos_:<8} | {children}")
```

TEXT	DEP	HEAD TEXT	HEAD POS	CHILDREN
Apple	nsubj	looking	VERB	[]
is	aux	looking	VERB	[]
looking	ROOT	looking	VERB	['Apple', 'is', 'at']
at	prep	looking	VERB	['buying']
buying	pcomp	at	ADP	['startup']
U.K.	compound	startup	NOUN	[]
startup	dobj	buying	VERB	['U.K.', 'for']
for	prep	startup	NOUN	['billion']
\$	quantmod	billion	NUM	[]
1	compound	billion	NUM	[]
billion	pobj	for	ADP	['\$', '1']

Phần 4: Duyệt cây phụ thuộc để trích xuất thông tin

4.1. Bài toán: Tìm chủ ngữ và tân ngữ của một động từ

```
text = "The cat chased the mouse and the dog watched them."
doc = nlp(text)
for token in doc:
    # Chỉ tìm các động từ
    if token.pos_ == "VERB":
        verb = token.text
        subject = ""

        obj = ""
        # Tìm chủ ngữ (nsubj) và tân ngữ (dobj) trong các con của động từ
        for child in token.children:
            if child.dep_ == "nsubj":
                subject = child.text
            if child.dep_ == "dobj":
                obj = child.text

        if subject and obj:
            print(f"Found Triplet: ({subject}, {verb}, {obj})")
```

```
text = "The big, fluffy white cat is sleeping on the warm mat."
doc = nlp(text)
for token in doc:
    # Chỉ tìm các danh từ
    if token.pos_ == "NOUN":
        adjectives = []
```

```
# Tìm các tính từ bổ nghĩa (amod) trong các con của danh từ
for child in token.children:
    if child.dep_ == "amod":
        adjectives.append(child.text)
if adjectives:
    print(f"Danh từ '{token.text}' được bổ nghĩa bởi các tính từ: {adjectives}")
```

▼ Phần 5: Bài tập tự luyện

▼ Bài 1: Tìm động từ chính của câu

- Ý tưởng: Duyệt qua danh sách token để tìm ROOT token

```
def find_main_verb(doc):
    for token in doc:
        if token.dep_ == "ROOT":
            return token
    return None # Trả về None nếu không tìm thấy ROOT (trường hợp hiếm)

# Ví dụ sử dụng hàm
text = "Apple is looking at buying U.K. startup for $1 billion"
doc = nlp(text)
main_verb = find_main_verb(doc)
if main_verb:
    print(f"Động từ chính của câu là: {main_verb.text}")
else:
    print("Không tìm thấy động từ chính.")

text_2 = "The quick brown fox jumps over the lazy dog."
doc_2 = nlp(text_2)
main_verb_2 = find_main_verb(doc_2)
if main_verb_2:
    print(f"Động từ chính của câu là: {main_verb_2.text}")
else:
    print("Không tìm thấy động từ chính.")
```

Động từ chính của câu là: looking
Động từ chính của câu là: jumps

▼ Bài 2: Trích xuất các cụm danh từ (Noun Chunks)

Ý tưởng:

- duyet qua từng token trong câu, xác định các token có khả năng là đầu của một cụm danh từ (danh từ, danh từ riêng, hoặc số).
- mở rộng cụm danh từ đó về phía bên trái để bao gồm các từ bổ nghĩa liên quan.
- sau khi tạo ra cụm danh từ, đánh dấu các chỉ mục của token đã được bao phủ để tránh trùng lặp.

```
def custom_noun_chunks(doc):
    chunks = []
    # Set để theo dõi các chỉ mục token đã được bao phủ bởi một cụm danh từ
    covered_indices = set()

    for token in doc:
        # Bỏ qua token nếu nó đã là một phần của cụm danh từ khác
        if token.i in covered_indices:
            continue

        # Tiêu chí cho một head tiềm năng của cụm danh từ:
        # - Là danh từ (NOUN)
        # - Là danh từ riêng (PROPN) và không phải là một thành phần ghép (compound modifier)
        # - Là số (NUM) đóng vai trò định lượng (quantmod) hoặc đối tượng của giới từ (pobj của ADP)
        is_potential_head = (
            (token.pos_ == "NOUN") or
            (token.pos_ == "PROPN" and token.dep_ != "compound") or
            (token.pos_ == "NUM" and (token.dep_ == "quantmod" or (token.dep_ == "pobj" and token.head.pos_ == "ADP")))
        )

        if is_potential_head:
            # Khởi tạo ranh giới cụm với token head
            chunk_start_idx = token.i

            # 'potential_chunk_start_candidate' giúp theo dõi ngược các phụ thuộc để mở rộng sang trái
            potential_chunk_start_candidate = token

            # Vòng lặp để mở rộng sang trái
            while potential_chunk_start_candidate.i > 0:
                left_token = doc[potential_chunk_start_candidate.i - 1]

                # Kiểm tra nếu 'left_token' là một từ bổ nghĩa và head của nó là:
                # - 'potential_chunk_start_candidate' (đối với các từ bổ nghĩa chuỗi như "New York")
                # - HOẶC 'token' gốc (đối với các từ bổ nghĩa trực tiếp như "The quick brown fox")
                is_modifier = left_token.dep_ in ["det", "amod", "compound", "nummod", "poss", "case", "quantmod"]
                is_connected = (left_token.head == potential_chunk_start_candidate or left_token.head == token)

                if is_modifier and is_connected:
                    chunk_start_idx = left_token.i
                    potential_chunk_start_candidate = left_token # Tiếp tục mở rộng từ token bên trái mới này
                else:
                    # Dừng mở rộng nếu không phải là từ bổ nghĩa hoặc không được kết nối
                    break
```

```
# Tạo một spaCy Span cho cụm danh từ
current_chunk = doc[chunk_start_idx : token.i + 1]
chunks.append(current_chunk.text)

# Đánh dấu tất cả các token trong cụm danh từ mới này là đã được bao phủ
for k in range(chunk_start_idx, token.i + 1):
    covered_indices.add(k)

return chunks

# --- Ví dụ sử dụng hàm ---

# Ví dụ 1
text_1 = "Apple is looking at buying U.K. startup for $1 billion"
doc_1 = nlp(text_1)
print(f"Câu: \"{text_1}\"")
print("Các cụm danh từ tùy chỉnh:", custom_noun_chunks(doc_1))
print("Các cụm danh từ của spaCy (.noun_chunks):", [chunk.text for chunk in doc_1.noun_chunks])
print("-" * 30)

# Ví dụ 2
text_2 = "The quick brown fox jumps over the lazy dog."
doc_2 = nlp(text_2)
print(f"Câu: \"{text_2}\"")
print("Các cụm danh từ tùy chỉnh:", custom_noun_chunks(doc_2))
print("Các cụm danh từ của spaCy (.noun_chunks):", [chunk.text for chunk in doc_2.noun_chunks])
print("-" * 30)

# Ví dụ 3 (có cụm danh từ phức tạp hơn: "The United States of America")
text_3 = "The United States of America is a large country."
doc_3 = nlp(text_3)
print(f"Câu: \"{text_3}\"")
print("Các cụm danh từ tùy chỉnh:", custom_noun_chunks(doc_3))
print("Các cụm danh từ của spaCy (.noun_chunks):", [chunk.text for chunk in doc_3.noun_chunks])
print("-" * 30)

# Ví dụ 4 (có tính từ)
text_4 = "A beautiful sunny day."
doc_4 = nlp(text_4)
print(f"Câu: \"{text_4}\"")
print("Các cụm danh từ tùy chỉnh:", custom_noun_chunks(doc_4))
print("Các cụm danh từ của spaCy (.noun_chunks):", [chunk.text for chunk in doc_4.noun_chunks])
print("-" * 30)
```

```
Câu: "Apple is looking at buying U.K. startup for $1 billion"
Các cụm danh từ tùy chỉnh: ['Apple', 'U.K. startup', '$1 billion']
Các cụm danh từ của spaCy (.noun_chunks): ['Apple', 'U.K. startup']
-----
Câu: "The quick brown fox jumps over the lazy dog."
Các cụm danh từ tùy chỉnh: ['The quick brown fox', 'the lazy dog']
Các cụm danh từ của spaCy (.noun_chunks): ['The quick brown fox', 'the lazy dog']
-----
Câu: "The United States of America is a large country."
Các cụm danh từ tùy chỉnh: ['The United States', 'America', 'a large country']
Các cụm danh từ của spaCy (.noun_chunks): ['The United States', 'America', 'a large country']
-----
Câu: "A beautiful sunny day."
Các cụm danh từ tùy chỉnh: ['A beautiful sunny day']
Các cụm danh từ của spaCy (.noun_chunks): ['A beautiful sunny day']
-----
```

▼ Bài 3: Tìm đường đi ngắn nhất trong cây

Ý tưởng:

- Bắt đầu từ token hiện tại: Hàm khởi tạo `current_token` bằng token mà bạn muốn tìm đường đi đến ROOT.
- Duyệt và thêm vào đường đi: kiểm tra xem `current_token` đã phải là ROOT chưa(`current_token.dep_ != "ROOT"`)
 - Nếu chưa phải là ROOT, nó thêm `current_token` vào danh sách `path`.
 - Sau đó, cập nhật `current_token` lên `current_token.head` (chính là PARENT của token hiện tại), để di chuyển lên một cấp trong cây phụ thuộc.
- Kết thúc và đảo ngược: Vòng lặp dừng lại khi `current_token` là ROOT. Sau đó, token ROOT cũng được thêm vào `path`.

```
def get_path_to_root(token):
    path = []
    current_token = token
    while current_token.dep_ != "ROOT" and current_token != current_token.head:
        path.append(current_token)
        current_token = current_token.head
    path.append(current_token) # Thêm token ROOT vào cuối đường đi
    path.reverse() # Đảo ngược danh sách để đường đi bắt đầu từ ROOT
    return path

# --- Ví dụ sử dụng hàm ---

text = "The quick brown fox jumps over the lazy dog."
doc = nlp(text)

# Lấy một token bất kỳ, ví dụ 'fox'
fox_token = doc[3] # 'fox' là token thứ 4 (index 3)

path_to_root = get_path_to_root(fox_token)
# Sửa lỗi: Sử dụng path_to_root[0].text để lấy ROOT token từ đường đi
print(f"Đường đi từ '{fox_token.text}' đến ROOT ('{path_to_root[0].text}'):")
print([t.text for t in path_to_root])

print("\n")
```

```
# Lấy một token khác, ví dụ 'dog'
dog_token = doc[8] # 'dog' là token thứ 9 (index 8)

path_to_root_dog = get_path_to_root(dog_token)
# Sửa lỗi: Sử dụng path_to_root_dog[0].text để lấy ROOT token từ đường đi
print(f"Đường đi từ '{dog_token.text}' đến ROOT ('{path_to_root_dog[0].text}'):")
print([t.text for t in path_to_root_dog])
```

Đường đi từ 'fox' đến ROOT ('jumps'):
['jumps', 'fox']

Đường đi từ 'dog' đến ROOT ('jumps'):
['jumps', 'over', 'dog']